

Assignment-16.4

Gujja Pranitha

2303A52171

Batch 41

Task 1: Hospital Management Database

PROMPT:

Create a SQL database for a hospital management scenario by defining three tables: doctors, patients, and appointments. Insert sample data into each table and write queries to join doctors with patients through appointments, display appointment details with doctor names, and calculate the number of unique patients handled by each doctor.

CODE:

```
1  -- SQLite
2  -- CREATE TABLES
3  CREATE TABLE doctors (
4      doctor_id INTEGER PRIMARY KEY,
5      first_name TEXT,
6      last_name TEXT
7  );
8  CREATE TABLE patients (
9      patient_id INTEGER PRIMARY KEY,
10     first_name TEXT,
11     last_name TEXT
12 );
13 CREATE TABLE appointments (
14     appointment_id INTEGER PRIMARY KEY,
15     doctor_id INTEGER,
16     patient_id INTEGER,
17     appointment_date DATE
18 );
19 -- INSERT DATA
20 INSERT INTO doctors VALUES (1, 'John', 'Doe');
21 INSERT INTO doctors VALUES (2, 'Jane', 'Smith');
22 INSERT INTO patients VALUES (1, 'Alice', 'Johnson');
23 INSERT INTO patients VALUES (2, 'Bob', 'Brown');
24 INSERT INTO appointments VALUES (1, 1, 1, '2024-07-01');
25 INSERT INTO appointments VALUES (2, 1, 2, '2024-07-02');
26 INSERT INTO appointments VALUES (3, 2, 1, '2024-07-03');
27 -- QUERY 1
28 SELECT
29     a.appointment_id,
30     d.first_name AS doctor_first_name,
31     d.last_name AS doctor_last_name,
32     p.first_name AS patient_first_name,
33     p.last_name AS patient_last_name,
34     a.appointment_date
35 FROM appointments a
```

```

36 JOIN doctors d ON a.doctor_id = d.doctor_id
37 JOIN patients p ON a.patient_id = p.patient_id;
38 -- QUERY 2
39 SELECT
40     a.appointment_id,
41     d.first_name,
42     d.last_name,
43     a.appointment_date
44 FROM appointments a
45 JOIN doctors d ON a.doctor_id = d.doctor_id;
46 -- QUERY 3
47 SELECT
48     d.doctor_id,
49     d.first_name,
50     d.last_name,
51     COUNT(DISTINCT a.patient_id) AS patient_count
52 FROM appointments a
53 JOIN doctors d ON a.doctor_id = d.doctor_id
54 GROUP BY d.doctor_id, d.first_name, d.last_name;

```

OUTPUT:

appointment_id	doctor_first_name	doctor_last_name	patient_first_name	patient_last_name	appointment
1	John	Doe	Alice	Johnson	2024-07-01
2	John	Doe	Bob	Brown	2024-07-02
3	Jane	Smith	Alice	Johnson	2024-07-03

SQL ▾	< 1 / 1 > 1 - 3 of 3				
appointment_id	first_name	last_name	appointment_date		
1	John	Doe	2024-07-01		
2	John	Doe	2024-07-02		
3	Jane	Smith	2024-07-03		

SQL ▾	< 1 / 1 > 1 - 2 of 2				
doctor_id	first_name	last_name	patient_count		
1	John	Doe	2		
2	Jane	Smith	1		

JUSTIFICATION:

The code first creates tables for doctors, patients, and appointments, then inserts sample records. The first query joins all three tables to show complete appointment details including doctor and patient names. The second query lists appointments along with doctor information. The third query groups data by doctor and counts distinct patients to show how many unique patients each doctor has treated.

Task 2: Real-Time Applica on: Online Shopping Database

PROMPT:

Design a database for an online shopping system by creating tables for Orders, OrderDetails, and Products. Insert sample data and write SQL queries to retrieve order details for a specific user, identify the top-selling product based on quantity sold, and calculate total revenue for a given time period.

CODE:

```
Welcome X exam.py -- SQLite Untitled-1 -- SQLite Untitled-2
1 -- SQLite
2 -- CREATE TABLES
3 CREATE TABLE Orders (
4     order_id INTEGER PRIMARY KEY,
5     user_id INTEGER,
6     order_date DATETIME
7 );
8 CREATE TABLE OrderDetails (
9     order_detail_id INTEGER PRIMARY KEY,
10    order_id INTEGER,
11    product_id INTEGER,
12    quantity INTEGER,
13    price REAL
14 );
15 CREATE TABLE Products (
16     product_id INTEGER PRIMARY KEY,
17     product_name TEXT
18 );
19 -- INSERT DATA
20 INSERT INTO Orders VALUES (1, 1, '2026-03-18 13:18:04');
21
22 INSERT INTO OrderDetails VALUES (1, 1, 1, 1, 999.99);
23 INSERT INTO OrderDetails VALUES (2, 1, 2, 2, 29.99);
24
25 INSERT INTO Products VALUES (1, 'Laptop');
26 INSERT INTO Products VALUES (2, 'Mouse');
27
28 -- QUERY 1: Order details for a user
29 SELECT
30     o.order_id,
31     o.order_date,
32     od.product_id,
33     od.quantity,
34     od.price
35 FROM Orders o
36 JOIN OrderDetails od ON o.order_id = od.order_id
37 WHERE o.user_id = 1;
38
39 -----
40 -- QUERY 2: Top selling product
41 SELECT
42     p.product_name,
43     SUM(od.quantity) AS total_quantity_sold
44 FROM Products p
45 JOIN OrderDetails od ON p.product_id = od.product_id
46 GROUP BY p.product_id
47 ORDER BY total_quantity_sold DESC
48 LIMIT 1;
49
50 -----
51 -- QUERY 3: Total revenue for a month
52 SELECT
53     SUM(od.quantity * od.price) AS total_revenue
54 FROM Orders o
55 JOIN OrderDetails od ON o.order_id = od.order_id
56 WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

OUTPUT:

order_id	order_date	product_id	quantity	price
1	2026-03-18 13:18:04	1	1	999.99
1	2026-03-18 13:18:04	2	2	29.99

SQL 1 / 1 1 - 1 of 1

product_name	total_quantity_sold
Mouse	2

SQL 1 / 1 1 - 1 of 1

total_revenue
NULL

JUSTIFICATION:

The code creates three tables representing orders, their details, and products, then inserts sample data. The first query joins Orders and OrderDetails to display all products purchased by a specific user. The second query aggregates quantities from OrderDetails and joins with Products to find the most sold product. The third query calculates total revenue by multiplying quantity and price, filtered by a specific month.

Task 3: Library Database

PROMPT:

Create a library management database by defining tables for Books, Members, and Loans. Insert sample data and write SQL queries to display details of borrowed books along with member information, and to calculate how many books each member has borrowed.

CODE:

```
exam.py x  -- SQLite Untitled-1  -- SQLite Untitled-2  -- SQLite Untitled-3  [ ]
-- SQLite
-- CREATE TABLES
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY,
    title TEXT,
    author TEXT
);
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY,
    first_name TEXT,
    last_name TEXT
);
CREATE TABLE Loans (
    loan_id INTEGER PRIMARY KEY,
    book_id INTEGER,
    member_id INTEGER,
    loan_date DATE
);
-- INSERT DATA
INSERT INTO Books VALUES (1, 'The Great Gatsby', 'F. Scott Fitzgerald');
INSERT INTO Books VALUES (2, 'To Kill a Mockingbird', 'Harper Lee');
INSERT INTO Members VALUES (1, 'John', 'Doe');
INSERT INTO Members VALUES (2, 'Jane', 'Smith');
INSERT INTO Loans VALUES (1, 1, 1, '2024-05-01');
INSERT INTO Loans VALUES (2, 2, 2, '2024-05-15');
-----
-- QUERY 1: Book loans with member details
SELECT
    b.title,
    b.author,
    m.first_name AS FirstName,
    m.last_name AS LastName,
    l.loan_date AS LoanDate
FROM Loans l
```

```

35 JOIN Books b ON l.book_id = b.book_id
36 JOIN Members m ON l.member_id = m.member_id;
37 -----
38 -- QUERY 2: (Same output again as shown)
39 SELECT
40     b.title,
41     b.author,
42     m.first_name AS FirstName,
43     m.last_name AS LastName,
44     l.loan_date AS LoanDate
45 FROM Loans l
46 JOIN Books b ON l.book_id = b.book_id
47 JOIN Members m ON l.member_id = m.member_id;
48 -----
49 -- QUERY 3: Count books loaned per member
50 SELECT
51     m.first_name AS FirstName,
52     m.last_name AS LastName,
53     COUNT(l.book_id) AS BooksLoaned
54 FROM Loans l
55 JOIN Members m ON l.member_id = m.member_id
56 GROUP BY m.member_id, m.first_name, m.last_name;

```

OUTPUT:

title	author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

title	author	FirstName	LastName	LoanDate
The Great Gatsby	F. Scott Fitzgerald	John	Doe	2024-05-01
To Kill a Mockingbird	Harper Lee	Jane	Smith	2024-05-15

FirstName	LastName	BooksLoaned
John	Doe	1
Jane	Smith	1

JUSTIFICATION:

The code creates three tables to store book details, member information, and loan records. After inserting sample data, the first two queries join all tables to show which member borrowed which book and the loan date. The final query groups the data by member and counts the number of books each member has borrowed, providing a summary of borrowing activity.

Task 4: Optimize Queries

PROMPT:

Create a simple database for managing books by defining a Books table with an availability status. Insert sample data and write SQL queries to retrieve all available books and demonstrate a query that returns no results when conditions are not met.

CODE:

```
-- SQLite Untitled-1 • -- SQLite Untitled-2 • -- SQLite Untitled-3 • -- SQLite Untitled-4 •
-- SQLite
-- CREATE TABLE
CREATE TABLE Books (
  book_id INTEGER PRIMARY KEY,
  title TEXT,
  available INTEGER
);

-- INSERT DATA
INSERT INTO Books VALUES (1, 'Book 1', 1);
INSERT INTO Books VALUES (2, 'Book 2', 0);
INSERT INTO Books VALUES (3, 'Book 3', 1);

-----
-- QUERY 1: Available books
SELECT title
FROM Books
WHERE available = 1;

-----
-- QUERY 2: Same result again (as shown)
SELECT title
FROM Books
WHERE available = 1;

-----
-- QUERY 3: No records (condition not matching)
SELECT title
FROM Books
WHERE available = 2;
```

OUTPUT:

title
Book 1
Book 3

SQL ▾ < 1 / 1 > 1 - 2 of 2

title
Book 1
Book 3

SQL ▾ < 1 / 1 > 1 - 0 of 0

JUSTIFICATION:

The code creates a Books table and stores data with an availability flag. The first two queries filter and display only the books that are available. The final query uses a condition that does not match any records, resulting in an empty output, which demonstrates how SQL handles unmatched conditions.

Task 5: University Course Registration

PROMPT:

Write SQL code in a single cell that creates three tables—students, courses, and registrations—inserts sample data for one student named Alice Smith enrolled in an “Introduction to Computer Science” course, and executes three queries: one to display Alice’s details, one that returns no results using a non-existing student, and one that shows the course name with the number of registered students.

CODE:

```
-- SQLite Untitled-3  ●  -- SQLite Untitled-4  ●  -- Create tables Untitled-5  ●
1  -- Create tables
2  CREATE TABLE students (
3      student_id INTEGER PRIMARY KEY,
4      first_name TEXT,
5      last_name TEXT,
6      email TEXT
7  );
8  CREATE TABLE courses (
9      course_id INTEGER PRIMARY KEY,
10     course_name TEXT
11 );
12 CREATE TABLE registrations (
13     registration_id INTEGER PRIMARY KEY,
14     student_id INTEGER,
15     course_id INTEGER
16 );
17 -- Insert sample data
18 INSERT INTO students (student_id, first_name, last_name, email)
19 VALUES (1, 'Alice', 'Smith', 'alice.smith@university.edu');
20 INSERT INTO courses (course_id, course_name)
21 VALUES (1, 'Introduction to Computer Science');
22 INSERT INTO registrations (registration_id, student_id, course_id)
23 VALUES (1, 1, 1);
24 -- 1. Get student details
25 SELECT first_name, last_name, email
26 FROM students
27 WHERE first_name = 'Alice' AND last_name = 'Smith';
28 -- 2. Query that returns no rows
29 SELECT *
30 FROM students
31 WHERE first_name = 'Bob' AND last_name = 'Smith';
32 -- 3. Course with registration count
33 SELECT c.course_name, COUNT(r.student_id) AS registration_count
34 FROM courses c
35 JOIN registrations r ON c.course_id = r.course_id
36 GROUP BY c.course_name;
```

OUTPUT:

first_name	last_name	email
Alice	Smith	alice.smith@university.edu

SQL ▾ < 1 / 1 > 1 - 0 of 0

SQL ▾ < 1 / 1 > 1 - 1 of 1

course_name	registration_count
Introduction to Computer Science	1

JUSTIFICATION:

The code first creates the required tables and inserts sample data to avoid “no such table” errors. Then, it runs three queries: the first uses a WHERE clause to filter and display Alice’s record, the second applies a condition that matches no rows to demonstrate an empty result, and the third uses a JOIN between courses and registrations along with GROUP BY and COUNT to calculate how many students are enrolled in each course.